

Übung 011 – Minimal API: Kursverwaltung

TECHNIK

.NET 10, ASP.NET Core,
EF Core mit SQLite

UMFANG

eine Datei `Program.cs`,
sechs Endpunkte

HILFSMITTEL

Doku ja, **AI nein**

TEIL 2

drei Änderungen, 45
Minuten

Worum es geht

Sie schreiben eine kleine REST-API zur Verwaltung von Kursen – **von Grund auf**, Endpunkt für Endpunkt. Die Bauweise ist bewusst schlicht: **alles in `Program.cs`**, der Datenbankzugriff und die Validierung stehen direkt im Endpunkt. Keine Schichten, keine Services, keine Repositories.

Das ist kein Versehen, sondern der Ausgangspunkt. An genau dieser Datei wird sich in Teil 2 zeigen, wo diese Bauweise an ihre Grenzen kommt – und daraus folgen die nächsten Stände.

Ohne AI. Sie dürfen die Microsoft-Dokumentation und die Präsentation *02_Minimal_Api* verwenden. Die fertige Vorlage im Ordner `011_Minimal_Api` sehen Sie erst an, wenn Sie feststecken – und dann nur die Stelle, an der Sie hängen.

Teil 1 – Die Anwendung schreiben

1. Projekt anlegen

```
dotnet new web -n Courses.Api
cd Courses.Api
dotnet add package Microsoft.EntityFrameworkCore.Sqlite
```

- Die Anwendung läuft auf `http://localhost:5080` (in `Properties/launchSettings.json` einstellen).
- Datenbank ist die SQLite-Datei `courses.db`. Sie wird beim Start mit `Database.EnsureCreated()` angelegt, falls sie fehlt – Migrationen gibt es noch nicht.
- Der `DbContext` heißt `CourseDb`, wird mit `AddDbContext` registriert und stellt ein `DbSet<Course> Courses` bereit.

Legen Sie das Projekt innerhalb des Repositorys `wmc-kurse` an, gilt dort die zentrale Paketverwaltung: In der `.csproj` darf dann kein `Version="..."` stehen, sonst meldet der Build `NU1008`. Details im README des Repositorys.

2. Die Entity `Course`

Property	Typ	Bedeutung
<code>Id</code>	<code>int</code>	Primärschlüssel, vergibt die Datenbank
<code>Title</code>	<code>string</code>	Titel des Kurses, eindeutig
<code>Trainer</code>	<code>string</code>	Name der Trainerin oder des Trainers
<code>PriceEuro</code>	<code>decimal</code>	Kurspreis in Euro
<code>Seats</code>	<code>int</code>	Anzahl der Plätze
<code>EnrolledCount</code>	<code>int</code>	Anzahl der bisherigen Anmeldungen
<code>StartsOn</code>	<code>DateOnly</code>	Kursbeginn

Eine einfache Klasse mit öffentlichen Gettern und Settern genügt. Sie wird direkt als Request- und als Response-Body verwendet – eigene DTOs gibt es in diesem Stand noch nicht.

3. Die sechs Endpunkte

Verb	Route	Verhalten	Antwort
GET	/courses	alle Kurse, aufsteigend nach <code>StartsOn</code> sortiert	200
GET	/courses/{id:int}	ein Kurs	200 · 404
POST	/courses	Kurs anlegen; <code>EnrolledCount</code> wird auf 0 gesetzt, egal was der Client schickt	201 mit <code>Location</code> · 400
PUT	/courses/{id:int}	Kurs ändern: Titel, Trainer, Preis, Plätze, Beginn. <code>EnrolledCount</code> bleibt unverändert.	200 · 400 · 404
DELETE	/courses/{id:int}	Kurs löschen	204 · 404
POST	/courses/{id:int}/enrol	eine Person anmelden, <code>EnrolledCount</code> um eins erhöhen	200 · 404 · 409

- `{id:int}` ist eine Routen-Constraint: GET /courses/abc trifft die Route gar nicht und ergibt 404.
- POST /courses antwortet mit `Results.Created`; der `Location`-Header zeigt auf /courses/{id} des neuen Kurses.
- Alle Endpunkte arbeiten asynchron (`ToListAsync`, `FindAsync`, `SaveChangesAsync`).

4. Validierung beim Anlegen und Ändern

Beide Endpunkte – POST und PUT – prüfen **dieselben** Regeln. Sind eine oder mehrere verletzt, antwortet der Endpunkt mit `Results.ValidationProblem(errors)`, also 400 mit **allen** Feldfehlern auf einmal, nicht nur dem ersten.

Feld	Regel	Meldung
title	nicht leer	Titel darf nicht leer sein.
	höchstens 120 Zeichen	Titel darf höchstens 120 Zeichen haben.
	eindeutig; beim Ändern zählt der Kurs selbst nicht mit	Ein Kurs mit diesem Titel existiert bereits.
trainer	nicht leer	Trainer darf nicht leer sein.
priceEuro	nicht negativ	Preis darf nicht negativ sein.
seats	zwischen 1 und 200	Plätze müssen zwischen 1 und 200 liegen.
startsOn	liegt nach dem heutigen Tag	Kursbeginn muss in der Zukunft liegen.

Die Feldnamen der Fehler sind die JSON-Namen (`title`, nicht `Title`). Beim Titel genügt die erste verletzte Regel.

5. Anmelden

POST /courses/{id:int}/enrol prüft in dieser Reihenfolge:

1. Gibt es den Kurs nicht → **404**.
2. Liegt der Beginn heute oder in der Vergangenheit → **409** mit „Der Kurs hat bereits begonnen.“
3. Ist `EnrolledCount` bereits gleich `Seats` → **409** mit „Der Kurs ist ausgebucht.“
4. Sonst `EnrolledCount` um eins erhöhen, speichern → **200** mit dem Kurs.

6. Ausprobieren mit einer .http -Datei

Legen Sie `courses.http` neben das Projekt und schreiben Sie für jeden Fall eine Anfrage – Visual Studio, VS Code (REST Client) und Rider schicken sie per „Send Request“ ab:

```
@host = http://localhost:5080

### Neuer Kurs -> 201 mit Location-Header
POST {{host}}/courses
Content-Type: application/json

{ "title": "Blazor Grundlagen", "trainer": "Koeck", "priceEuro": 390,
  "seats": 12, "startsOn": "2026-11-03" }
```

Mindestens abzudecken: jeder Statuscode aus der Endpunkt-Tabelle, ein `POST`, bei dem alle Regeln gleichzeitig verletzt sind, `GET /courses/abc`, zweimal `DELETE` auf dieselbe Id, und so viele Anmeldungen, bis der Kurs ausgebucht ist.

Fertig ist Teil 1, wenn `dotnet build` ohne Warnungen durchläuft und jede Anfrage in Ihrer `courses.http` den erwarteten Statuscode liefert.

Teil 2 – Drei Änderungswünsche

Jetzt kommen Anforderungen, wie sie in echten Projekten kommen. **Von Hand, ohne AI, Zeitrahmen 45 Minuten für alle drei.** Bauen Sie nichts vorher um – arbeiten Sie mit der Datei, wie sie ist.

Nach jeder Änderung beantworten Sie die Frage darunter **schriftlich**, in zwei, drei Sätzen. Die Antwort ist der eigentliche Ertrag dieser Übung, nicht der Code.

A • Neue Regel. Ein Kurs darf nur angelegt werden, wenn der Preis pro Platz (`PriceEuro / Seats`) unter 500 € liegt.

Danach: An wie vielen Stellen mussten Sie die Regel hinschreiben?

B • Zweiter Zugang. Neben der API soll eine Blazor-Seite Kurse anlegen können – mit denselben Regeln.

Danach: Wie kommen Sie an die Validierung heran, ohne HTTP zu sprechen?

C • Ein Test. Schreiben Sie einen Unit-Test für „Kurs ist ausgebucht → Anmeldung wird abgelehnt“.

Danach: Was brauchen Sie alles, damit dieser Test läuft?

Wer nicht fertig wird, hat trotzdem gewonnen. **Wer alle drei gelöst hat, hat die Antwort darauf, warum die nächsten Stände kommen. Wer sie nicht gelöst hat, erst recht.**

Zum Schluss – Vergleich mit der Vorlage

Öffnen Sie jetzt `011_Minimal_Api/Program.cs` im Repository und legen Sie es neben Ihre Lösung.

In der Vorlage steckt ein absichtlicher Fehler, wie er beim Kopieren von Code entsteht: Dasselbe Eingabedatum wird an einer Stelle abgelehnt und an einer anderen angenommen. Finden Sie ihn – zuerst durch Lesen, dann mit einer Anfrage in Ihrer `courses.http`, die ihn zeigt. Und prüfen Sie, ob Ihre eigene Lösung denselben Fehler hat.

Zwei Fragen, die nach jedem Stand dieses Beispiels gestellt werden:

1. **Was ist an dieser Bauweise gut?** Für welches Projekt wäre genau diese eine Datei die richtige Antwort?
2. **Was tut weh?** Nennen Sie mindestens zwei Stellen, an denen Teil 2 gezeigt hat, wo es hakt.

Die Begriffe dazu – Host und Pipeline, Parameter-Binding, Routen-Constraints, `Results.*`, Change Tracking, `EnsureCreated` – stehen in der Präsentation *02_Minimal_Api*. Prüfungsrelevant ist ausschließlich, was im Wiki steht.